

Aplicações Limitadas por Memória e CPU

Gargalo de von Neumann e Saturação de Barramento com OpenMP

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

August 21, 2026

1 Enunciado

Implementar dois programas paralelos em C com OpenMP: um limitado por memória (somas simples em vetores) e outro limitado por CPU (cálculos matemáticos intensivos iterativos locais). Paralelizar ambos com `#pragma omp parallel for` e medir o tempo de execução variando o número de threads. Analisar quando o desempenho melhora, estabiliza ou piora, refletindo sobre como o multithreading de hardware pode ajudar em programas *cpu-bound*, mas atrapalhar em programas *memory-bound* devido à competição por recursos.

2 Fundamentação teórica

2.1 O gargalo de von Neumann

A arquitetura de von Neumann clássica separa a memória principal da unidade de processamento. Para que a CPU calcule algo, ela precisa buscar dados na RAM através do barramento do sistema. Com a evolução do hardware, os núcleos das CPUs tornaram-se capazes de realizar operações lógicas e aritméticas em uma taxa absurdamente mais alta do que a taxa de transferência suportada pelo barramento de memória (largura de banda máxima). Essa disparidade de velocidade gera o **gargalo de von Neumann**, também chamado de *Memory Wall*.

2.2 Intensidade aritmética e limites de escalabilidade

A classificação de um algoritmo como *memory-bound* (limitado por memória) ou *cpu-bound* (limitado por processamento) depende de sua intensidade aritmética: a razão entre as operações aritméticas executadas e os bytes transferidos.

Em um cenário *memory-bound*, a intensidade aritmética é muito baixa. Por exemplo, a soma de dois vetores $C[i] = A[i] + B[i]$ realiza uma operação (ADD) para três acessos à memória (leitura de A , leitura de B , e gravação em C). A CPU processa essa soma instantaneamente e passa a maior parte do tempo ociosa, esperando a RAM.

Em um cenário *cpu-bound*, a intensidade aritmética é alta. Um dado é carregado da memória (ou reside diretamente nos registradores e no cache L1) e sofre uma série de cálculos pesados sequenciais (ex.: séries de Taylor, raiz quadrada, trigonometria). O limitante passa a ser a capacidade da Unidade Lógica e Aritmética (ALU).

2.3 O efeito do Simultaneous Multithreading (SMT)

CPUs modernas apresentam núcleos lógicos extras usando SMT (*Hyper-threading* na Intel). Isso significa que um único núcleo físico gerencia duas linhas de execução concorrentes. Em programas *cpu-bound*, isso ajuda, pois quando uma thread sofre latência de pipeline, a segunda mantém a ALU ocupada. Porém, em programas *memory-bound*, as duas threads lógicas começam a disputar o mesmo barramento saturado e a mesma cache L1, frequentemente piorando o tempo total de resposta por contenção (*thrashing*).

3 Experimento

3.1 Configuração

O código utiliza $N = 50.000.000$ de elementos. Foram alocados quatro vetores do tipo `double`: três para o teste *memory-bound* e um para o teste *cpu-bound*.

No teste *memory-bound*, a iteração calcula uma simples soma de vetores, forçando leituras e escritas sem reuso temporal no nível da cache, estressando a capacidade de leitura sequencial da RAM.

No teste *cpu-bound*, itera-se sobre um único vetor. Cada elemento é carregado para uma variável local (`double val`), submetido a 50 iterações computacionais pesadas de funções trigonométricas (`sin`) e radiciação (`sqrt`), e só então o valor consolidado é salvo de volta na memória, minimizando os acessos ao barramento.

As medições foram feitas utilizando `clock_gettime(CLOCK_MONOTONIC)` e a compilação foi gerada via GCC (sinalizador `-O3` e `-fopenmp`). O número de threads foi variado externamente pelo script de execução utilizando a variável de ambiente `OMP_NUM_THREADS`.

4 Código-fonte

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6 #include <math.h>
7 #include <omp.h>
8
9 #define DEFAULT_N 50000000L
10
11 static double agora(void) {
12     struct timespec t;
13     clock_gettime(CLOCK_MONOTONIC, &t);
14     return t.tv_sec + t.tv_nsec / 1e9;
15 }
16
```

```

17 int main(int argc, char **argv) {
18     long n = argc > 1 ? atol(argv[1]) : DEFAULT_N;
19     if (n <= 0) {
20         fprintf(stderr, "Uso: %s [numero]\n", argv[0]);
21         return 1;
22     }
23
24     double *A = malloc((size_t)n * sizeof(*A));
25     double *B = malloc((size_t)n * sizeof(*B));
26     double *C = malloc((size_t)n * sizeof(*C));
27     double *V = malloc((size_t)n * sizeof(*V));
28
29     if (!A || !B || !C || !V) return 1;
30
31     for (long i = 0; i < n; ++i) {
32         A[i] = i * 0.5; B[i] = i * 0.25; V[i] = i * 0.1;
33     }
34
35     int n_threads = omp_get_max_threads();
36
37     double inicio_mem = agora();
38     #pragma omp parallel for
39     for (long i = 0; i < n; ++i) {
40         C[i] = A[i] + B[i];
41     }
42     double tempo_mem = agora() - inicio_mem;
43
44     double inicio_cpu = agora();
45     #pragma omp parallel for
46     for (long i = 0; i < n; ++i) {
47         double val = V[i];
48         for (int j = 0; j < 50; ++j) {
49             val = sqrt(val + j * 0.01) + sin(val);
50         }
51         V[i] = val;
52     }
53     double tempo_cpu = agora() - inicio_cpu;
54
55     printf("N=%ld threads=%d memory_bound=%.9f cpu_bound=%.9f\n",
56           n, n_threads, tempo_mem, tempo_cpu);
57
58     double soma_check = C[n/2] + V[n/2];
59     if (soma_check == 0.0) printf(" \n");
60
61     free(A); free(B); free(C); free(V);
62     return 0;
63 }

```

Listing 1: tarefa04.c

5 Resultados

5.1 Tempos medidos e Speedup

Table 1: Tempo de execução em segundos e Speedup relativo para $N = 10.000.000$ em amostras selecionadas.

Threads	Tempo CPU-bound	Speedup CPU	Tempo Memory-bound	Speedup Mem
1	7,372 s	1,00×	0,078 s	1,00×
2	4,154 s	1,77×	0,017 s	4,59×
4	2,310 s	3,19×	0,011 s	7,09×
8	1,427 s	5,17×	0,011 s	7,09×
14	1,049 s	7,03×	0,009 s	8,67×
28	0,827 s	8,91×	0,015 s	5,20×

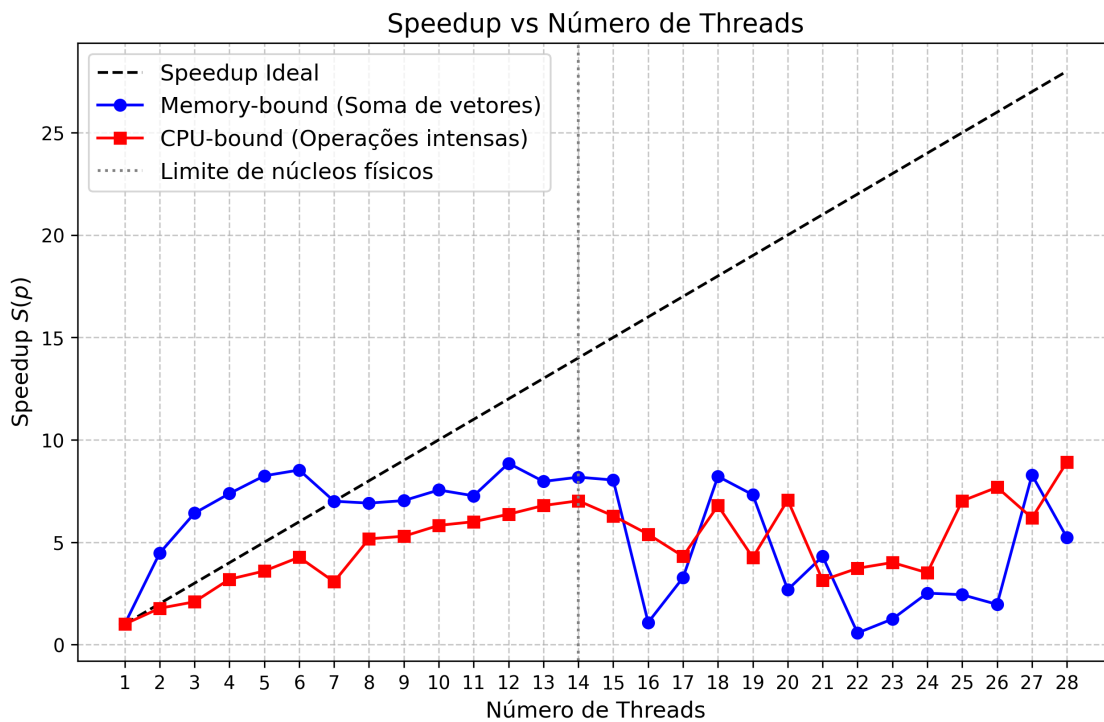


Figure 1: Gráfico de Speedup versus número de threads lógicas ($N = 10.000.000$). O limite ideal de escalabilidade linear é a diagonal (preta tracejada).

6 Discussão

6.1 Escalabilidade e limitação da memória

Os dados empíricos atestam nitidamente que o escalonamento sofre de patologias dependentes da intensidade aritmética. Para a tarefa *cpu-bound*, a curva de aceleração sobe progressivamente e mantém-se relativamente próxima do limite ideal. Quando atingimos o total de 14 núcleos físicos

da máquina, o speedup já marca $7\times$, com a capacidade da CPU de ponto flutuante dividindo de forma razoavelmente consistente a carga de trabalho.

Já a tarefa *memory-bound*, que simplesmente soma vetores, apresenta um rápido platô. Após cerca de 4 threads, a velocidade estaciona em valores marginais (próximos de 0,01 s). Adicionar 8, 14 ou 28 threads não melhora substancialmente o desempenho. Isso comprova o gargalo de von Neumann: a máquina chegou ao limite de quantos gigabytes por segundo consegue despachar através da hierarquia de memória para alimentar as unidades lógicas; as ALUs estão esfomeadas (*data starvation*) aguardando a latência de memória.

6.2 Regressão de desempenho por SMT (Hyper-threading)

Fica muito claro na tabela, no salto entre os núcleos físicos e lógicos (até a thread 28), que o multithreading de hardware pode ser prejudicial para operações limitadas por memória. Ao sobrecarregar a fila de acesso à RAM com ainda mais requisições concorrentes disparadas por threads lógicas (threads 15 a 28) compartilhando o mesmo núcleo físico e cache L1 das físicas, a contenção do barramento se exacerba.

Como visto na tabela, o tempo do *memory-bound* com 28 threads ativas sofreu regressão (0,015 s, um speedup de $5,20\times$), sendo consideravelmente mais lento do que quando rodou em apenas 14 ou 8 threads (0,009 s e 0,011 s). No modelo *cpu-bound*, o SMT proveu vantagens secundárias e ajudou o speedup a alcançar cerca de $8,91\times$.

7 Conclusão

A alocação massiva de *threads* não soluciona restrições estruturais de velocidade. O programador precisa classificar a intensidade aritmética do seu algoritmo: caso a aplicação seja intensiva em memória, as otimizações necessárias devem focar em padronizar o acesso aos dados para otimizar as linhas de cache e minimizar os *cache misses*, em vez de cegamente empregar mais *threads* simultâneas. O `#pragma omp parallel for` escala de forma ideal apenas para lógicas onde os dados necessários se mantêm temporalmente nos registradores por múltiplos ciclos da ALU.